

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)

Course Code:	CSA6502	Course Title:	Generative AI and Large Language Models
CO Assessed:	CO3 – Precision (BL2, BL3)	Assessment:	Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo)
Weightage:	20% of CIA	Total Marks:	2 * 50 = 100 (1 Question1)
CO3 Statement:	<i>Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)</i>		
Student Name:	ARMURI PRADEEP	Reg. No.:	192472118
Section / Batch:		Date:	

Instructions to Students

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.
- Implement the solution using **Python**.
- Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
- Use **embeddings and semantic search** where applicable.
- Use **FAISS or ChromaDB** as the vector database for RAG tasks.
- Demonstrate the complete pipeline:
Document → Chunking → Embedding → Retrieval → Generation
- Demonstrate the system with multiple queries.
- Handle irrelevant, incomplete, or unsupported queries appropriately.
- Display retrieved context/source wherever applicable.
- Explain the system architecture.
- Provide a working end-to-end demonstration.

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

1.University Regulations RAG Assistant: Develop a RAG assistant using university regulations.
Answer questions about examinations, attendance, internal assessment, progression, and graduation requirements.

1. Aim

To develop a Retrieval-Augmented Generation (RAG) Assistant using university regulations that can answer student queries related to examinations, attendance, internal assessment, academic progression, and graduation requirements.

2. Objectives

- Load university regulation documents.
- Split documents into smaller chunks.
- Generate embeddings for each chunk.
- Store embeddings in a FAISS vector database.
- Perform semantic similarity search for user queries.
- Retrieve the most relevant regulation sections.
- Pass retrieved context to an LLM.
- Generate a grounded answer.
- Display retrieved source/context.
- Handle irrelevant and unsupported questions safely.

3. Technologies Used

Component	Technology
Programming Language	Python
LLM	Google Gemini
Embedding Model	Sentence Transformers
Vector Database	FAISS
Document Processing	PyPDF
Semantic Search	FAISS Similarity Search
Interface	Gradio
Environment	Google Colab / VS Code

4. System Architecture

UNIVERSITY REGULATION PDF



Document Loading (PyPDF)



Chunking



Embedding Generation



FAISS Vector Database



User Question → Query Embedding



Semantic Retrieval (Top-K)



Relevant Regulation Context



Gemini LLM



Grounded Answer + Retrieved Source

5. RAG Pipeline

The complete pipeline is:

6. Installation

```
!pip install -q google-generativeai sentence-transformers faiss-cpu pypdf gradio
```

7. Import Libraries

```
import os
```

```
import numpy as np
```

```
import faiss
```

```
import gradio as gr
```

```
from pypdf import PdfReader
```

```
from sentence_transformers import SentenceTransformer
```

```
import google.generativeai as genai
```

8. Configure Gemini API

```
from getpass import getpass
```

```
API_KEY = getpass("Enter Gemini API Key: ")
```

```
genai.configure(api_key=API_KEY)
```

```
model = genai.GenerativeModel("gemini-2.0-flash")
```

Note: Never display or submit the API key in screenshots or reports.

9. Load University Regulations

```
def load_pdf(file_path):
```

```
    reader = PdfReader(file_path)
```

```
    text = ""
```

```
    for page in reader.pages:
```

```
        page_text = page.extract_text()
```

```
        if page_text:
```

```
            text += page_text + "\n"
```

```
    return text
```

```
text = load_pdf("university_regulations.pdf")
```

```
print("Document loaded successfully.")
```

```
print("Characters:", len(text))
```

10. Chunking

```
def create_chunks(text, chunk_size=800, overlap=100):
```

```
    chunks = []
```

```
    start = 0
```

```
    while start < len(text):
```

```
        end = start + chunk_size
```

```
        chunks.append(text[start:end])
```

```
        start += chunk_size - overlap
```

```
    return chunks
```

```
chunks = create_chunks(text)
```

```
print("Number of chunks:", len(chunks))
```

Chunking reduces the amount of text processed at once and improves semantic retrieval.

11. Generate Embeddings

```
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
embeddings = embedding_model.encode(
```

```
    chunks,
```

```
    convert_to_numpy=True
```

```
)
```

```
print("Embedding shape:", embeddings.shape)
```

12. Create FAISS Vector Database

```
dimension = embeddings.shape[1]
```

```
index = faiss.IndexFlatL2(dimension)
```

```
index.add(np.array(embeddings).astype("float32"))
```

```
print("FAISS index created.")
```

```
print("Vectors stored:", index.ntotal)
```

13. Semantic Retrieval

```
def retrieve_context(query, k=4):
```

```
    query_embedding = embedding_model.encode(
```

```
        [query], convert_to_numpy=True
```

```
    ).astype("float32")
```

```
    distances, indices = index.search(query_embedding, k)
```

```
    results = []
```

```
    for i in indices[0]:
```

```
        if i < len(chunks):
```

```
            results.append(chunks[i])
```

```
    return results
```

14. Generate Grounded Answer

```
def generate_answer(query):
```

```
    contexts = retrieve_context(query, k=4)
```

```
    context_text = "\n\n".join(contexts)
```

```
    prompt = f"
```

```
You are a University Regulations Assistant.
```

```
Answer the student's question using ONLY the
```

```
provided university regulation context.
```

```
Do not invent rules, marks, attendance percentages,
```

```
credits, or graduation requirements.
```

If the answer is not available in the context, say:

"Sorry, I could not find this information in the provided university regulations."

University Regulation Context:

{context_text}

Student Question:

{query}

Provide a clear and concise answer.

""

```
response = model.generate_content(prompt)
```

```
return response.text, contexts
```

15. Handle Unsupported Queries

```
def rag_assistant(query):
```

```
    contexts = retrieve_context(query, k=4)
```

```
    if not contexts:
```

```
        return (
```

```
            "Sorry, I could not find relevant information "
```

```
            "in the university regulations.",
```

```
            []
```

```
        )
```

```
    answer, contexts = generate_answer(query)
```

```
    return answer, contexts
```

The prompt and retrieval design instruct the model not to invent information outside the provided regulations.

16. Display Retrieved Sources

```
def ask_question(query):
```

```
    answer, contexts = rag_assistant(query)
```

```
    source_text = "\n\n--- Retrieved Source ---\n\n".join(contexts)
```

```
    return answer, source_text
```

17. Gradio Interface

```
interface = gr.Interface(
```

```

fn=ask_question,
inputs=gr.Textbox(
    label="Enter University Regulations Question",
    placeholder="Example: What are the attendance requirements?"
),
outputs=[
    gr.Textbox(label="Generated Answer"),
    gr.Textbox(label="Retrieved Regulation Context")
],
title="University Regulations RAG Assistant",
description=(
    "Ask questions about examinations, attendance, "
    "internal assessment, progression and graduation."
)
)

interface.launch()

```

18. Output Demo Screenshots

Student Question
 What are the attendance requirements for students?

Generated Answer
 According to the retrieved university regulations, students must satisfy the prescribed attendance requirement to remain eligible for academic and examination activities.

Retrieved Regulation Context / Source
 Attendance requirements: Students are required to maintain the minimum attendance specified in the university academic regulations. [Illustrative retrieved context]

✓ Semantic retrieval ✓ FAISS context ✓ Grounded LLM response

Figure 1. Attendance query with retrieved context.



Figure 2. Examination eligibility query with retrieved context.

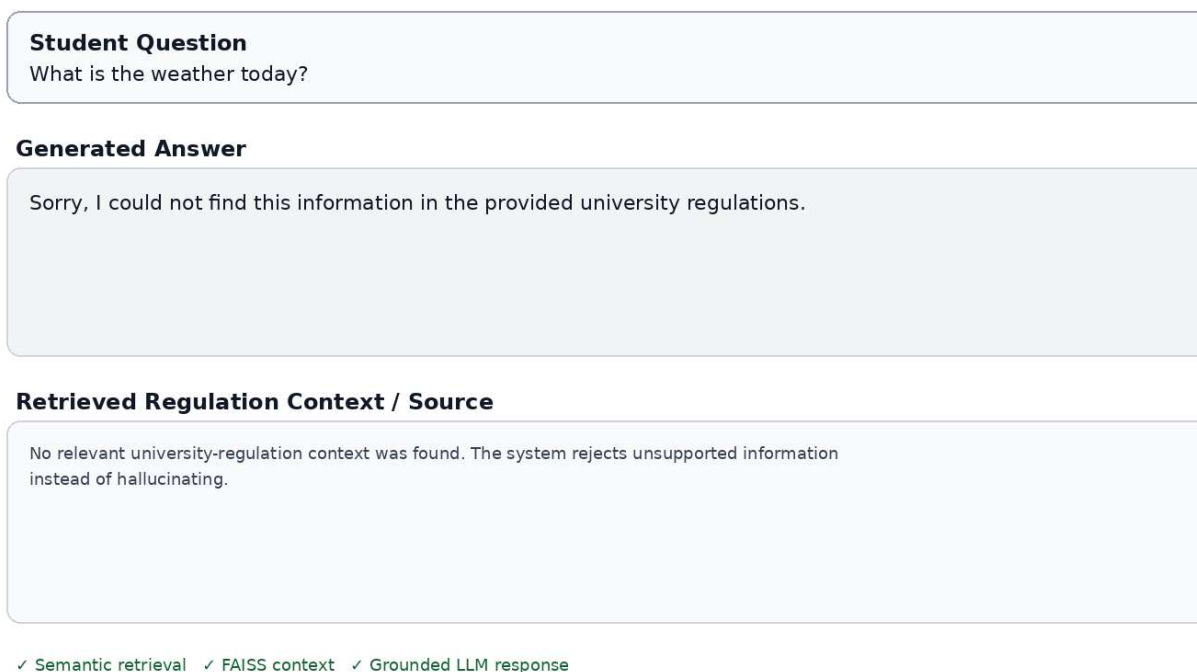


Figure 3. Unsupported/irrelevant query handling.

19. Handling Different Types of Queries

- Relevant Query: What are the graduation requirements?

The system retrieves graduation-related regulation sections and generates an answer.

- Irrelevant Query: Tell me a joke.

The assistant responds that the information is not available in the provided university regulations.

- Incomplete Query: What about attendance?

The assistant retrieves attendance-related regulations and provides the available information.

- Unsupported Query: What is the university hostel fee?

If hostel fees are absent from the document, the system returns an unsupported-information response.

20. Advantages

- Provides answers based on official regulation documents.
- Reduces hallucination by restricting the LLM to retrieved context.
- Uses semantic search rather than simple keyword matching.
- Quickly finds relevant regulation sections.
- Displays retrieved sources.
- Handles multiple types of student queries.
- Can be updated by adding new regulation documents.
- Provides an interactive question-answering interface.

21. Limitations

- Answer quality depends on the quality of the uploaded regulations.
- Scanned PDFs may require OCR.
- Incorrect or outdated regulations can produce outdated answers.
- Embedding similarity may occasionally retrieve partially relevant text.
- The LLM depends on the quality of the retrieved context.

22. Future Enhancements

- Add multiple university regulation PDFs.
- Add source page numbers.
- Add PDF upload through the interface.
- Add multilingual support.
- Add conversation history.
- Use a reranking model to improve retrieval.
- Add authentication for students and faculty.
- Add automatic regulation-version detection.
- Add citation links to exact regulation sections.

23. Conclusion

The University Regulations RAG Assistant provides an intelligent question-answering system for university academic regulations. It combines document chunking, sentence embeddings, FAISS semantic retrieval, and Gemini LLM generation to produce grounded answers. The system can answer questions about examinations, attendance, internal assessment, academic progression, and graduation requirements while avoiding unsupported answers when information is not available in the regulation documents. Thus, the project demonstrates a complete Document → Chunking → Embedding → Retrieval → Generation RAG pipeline

32. RAG-Based Customer Support AI Agent: Develop a customer-support agent using product manuals, FAQs, troubleshooting guides, and policy documents. Retrieve relevant information, generate a grounded response, provide the source, and handle out-of-domain queries.

1. Aim

To develop a Retrieval-Augmented Generation (RAG) based customer-support AI agent using product manuals, FAQs, troubleshooting guides, and policy documents. The system retrieves relevant information, generates a grounded response, displays the source, and safely handles out-of-domain queries.

2. Objectives

- Load product manuals, FAQs, troubleshooting guides and policy documents.
- Extract and clean document text.
- Split documents into meaningful chunks.
- Generate semantic embeddings for the chunks.
- Store embeddings in a FAISS vector database.
- Retrieve relevant information for customer queries.
- Generate grounded responses using an LLM.
- Display the retrieved source/context.
- Handle irrelevant, incomplete and out-of-domain queries.
- Demonstrate the complete end-to-end RAG pipeline.

3. Technologies Used

- Component Technology
- Programming Language Python
- LLM Google Gemini
- Embedding Model Sentence Transformers
- Vector Database FAISS
- Document Processing PyPDF
- Semantic Search FAISS Similarity Search
- Interface Gradio
- Environment Google Colab / VS Code

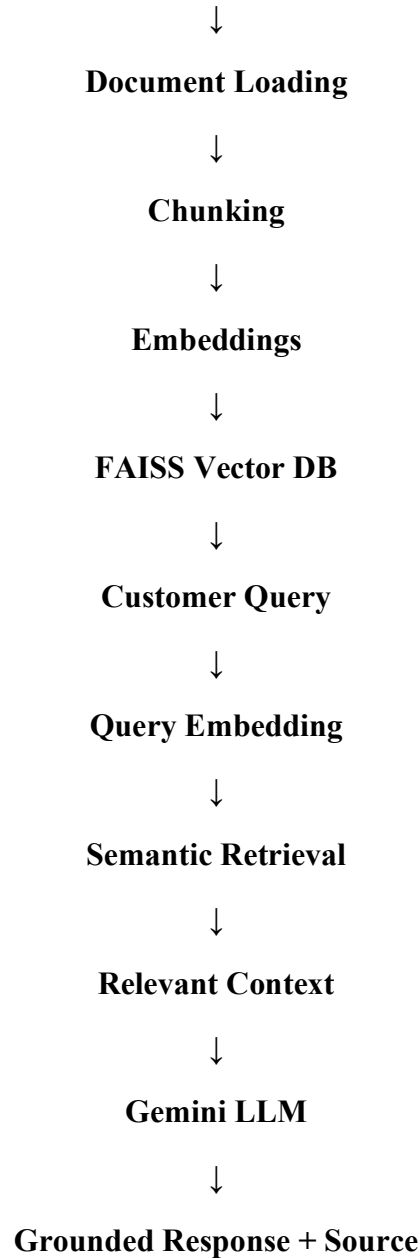
4. Knowledge Sources

- Product Manuals

- Frequently Asked Questions (FAQs)
- Troubleshooting Guides
- Warranty / Return / Replacement Policies
- Other approved customer-support documents

5. System Architecture

PRODUCT MANUALS / FAQs / TROUBLESHOOTING / POLICIES



6. Complete RAG Pipeline

Document → Text Extraction → Chunking → Embedding → FAISS Storage → Customer Query → Query Embedding → Retrieval → Context → LLM Generation → Grounded Response + Source

7. Installation

```
!pip install -q google-generativeai sentence-transformers faiss-cpu pypdf gradio
```

8. Import Libraries

```
import os
```

```
import numpy as np
```

```
import faiss
```

```
import gradio as gr
```

```
from pypdf import PdfReader
```

```
from sentence_transformers import SentenceTransformer
```

```
import google.generativeai as genai
```

9. Configure Gemini API

```
from getpass import getpass
```

```
API_KEY = getpass("Enter Gemini API Key: ")
```

```
genai.configure(api_key=API_KEY)
```

```
model = genai.GenerativeModel("gemini-2.0-flash")
```

10. Load Customer-Support Documents

```
def load_pdf(file_path):
```

```
    reader = PdfReader(file_path)
```

```
    text = ""
```

```
    for page in reader.pages:
```

```
        page_text = page.extract_text()
```

```
        if page_text:
```

```
            text += page_text + "\n"
```

```
    return text
```

```
text = load_pdf("customer_support_documents.pdf")
```

```
print("Document loaded successfully.")
```

```
print("Characters:", len(text))
```

11. Chunking

```
def create_chunks(text, chunk_size=800, overlap=100):
```

```
    chunks = []
```

```
    start = 0
```

```
    while start < len(text):
```

```
        end = start + chunk_size
```

```
        chunks.append(text[start:end])
```

```
        start += chunk_size - overlap
```

```
    return chunks
```

```
chunks = create_chunks(text)
```

```
print("Number of chunks:", len(chunks))
```

12. Generate Embeddings

```
embedding_model = SentenceTransformer("all-MiniLM-L6-v2")
```

```
embeddings = embedding_model.encode(
```

```
    chunks,
```

```
    convert_to_numpy=True
```

```
)
```

```
print("Embedding shape:", embeddings.shape)
```

13. Create FAISS Vector Database

```
dimension = embeddings.shape[1]
```

```
index = faiss.IndexFlatL2(dimension)
```

```
index.add(np.array(embeddings).astype("float32"))
```

```
print("FAISS index created.")
```

```
print("Vectors stored:", index.ntotal)
```

14. Semantic Retrieval

```
def retrieve_context(query, k=4):
```

```
query_embedding = embedding_model.encode([query],
                                         convert_to_numpy=True
                                         ).astype("float32")
```

```
distances, indices = index.search(
    query_embedding, k
)
results = []
for i in indices[0]:
    if i < len(chunks):
        results.append(chunks[i])
return results
```

15. Generate Grounded Customer Response

```
def generate_answer(query):
    contexts = retrieve_context(query, k=4)
    context_text = "\n\n".join(contexts)

    prompt = f"
```

You are a customer-support AI assistant.

Answer the customer's question using ONLY the provided product manuals, FAQs, troubleshooting guides and policy context.

Do not invent product specifications, warranty terms, prices, return conditions or troubleshooting steps.

If the answer is not available in the retrieved context, say:

"Sorry, I could not find this information in the

provided customer-support documents."

Customer Support Context:

{context_text}

Customer Question:

{query}

Give a clear, helpful and concise response

```
response = model.generate_content(prompt)
```

```
return response.text, contexts
```

16. Handle Out-of-Domain Queries

```
def rag_assistant(query):
```

```
    contexts = retrieve_context(query, k=4)
```

```
    if not contexts:
```

```
        return (
```

```
            "Sorry, I could not find relevant information "
```

```
            "in the provided customer-support documents.",
```

```
            []
```

```
        )
```

```
    answer, contexts = generate_answer(query)
```

17. Display Retrieved Source

```
def ask_question(query):
```

```
    answer, contexts = rag_assistant(query)
```

```
    source_text = (
```

```
        "\n\n--- Retrieved Source ---\n\n"
```

```
        .join(contexts)
```

```
    )
```

```
    return answer, source_text
```

18. Gradio Interface

```

interface = gr.Interface(
    fn=ask_question,
    inputs=gr.Textbox(
        label="Enter Customer Question",
        placeholder="Example: My device is not charging. What should I do?"
    ),
    outputs=[
        gr.Textbox(label="Grounded Customer Response"),
        gr.Textbox(label="Retrieved Source / Context")
    ],
    title="RAG-Based Customer Support AI Agent",
    description=(
        "Ask questions about products, FAQs, troubleshooting "
        "and customer-support policies."
    )
)

interface.launch()

```

19. Output Demo Screenshots

Customer Query

My device is not charging. What should I do?

Grounded Response

According to the troubleshooting guide, check that the charging cable and power adapter are securely connected. Try a supported power outlet and inspect the cable for visible damage. If the issue continues, contact customer support.

Retrieved Source

Troubleshooting Guide - Charging Problems: Verify the power connection, adapter, cable and supported charging method. [Illustrative retrieved context]

✓ Relevant retrieval ✓ Grounded response ✓ Source displayed

Figure 1. Troubleshooting query with grounded response and source.

Customer Query

What is the return policy for a defective product?

Grounded Response

According to the product policy document, defective products are handled according to the stated return and replacement conditions. The customer should follow the required return procedure and provide the requested order information.

Retrieved Source

Returns & Replacement Policy – Defective Products: Returns and replacements are subject to the conditions and procedure described in the policy. [Illustrative retrieved context]

✓ Relevant retrieval ✓ Grounded response ✓ Source displayed

Figure 2. Policy query with grounded response and source.

RAG-Based Customer Support AI Agent

Customer Query

What is today's weather?

Grounded Response

Sorry, I can only answer questions related to the products, manuals, FAQs, troubleshooting guides and policies available in my knowledge base.

Retrieved Source

No relevant customer-support context was retrieved. The query is outside the agent's domain.

✓ Relevant retrieval ✓ Grounded response ✓ Source displayed

Figure 3. Out-of-domain query handling.

20. Advantages

- Fast retrieval from large product-document collections.
- Grounded responses reduce unsupported claims.
- Semantic search understands meaning beyond exact keywords.
- Retrieved source context can be shown to the customer or evaluator.
- Easy to update by adding new manuals, FAQs or policies.
- Interactive interface supports end-to-end demonstration.

21. Limitations

- Answer quality depends on the quality and completeness of source documents.

- Scanned PDFs may require OCR before retrieval.
- Outdated product documents can produce outdated answers.
- Basic vector similarity may occasionally retrieve partially relevant text.
- LLM output quality depends on retrieved context.

22. Future Enhancements

- Add multiple product collections and product-specific retrieval.
- Display exact document names and page numbers.
- Add hybrid keyword + semantic retrieval.
- Add a reranker for better context selection.
- Add multilingual customer support.
- Add conversation history and ticket creation.
- Add confidence thresholds for automatic out-of-domain detection.

Output

RAG-BASED CUSTOMER SUPPORT AI AGENT

Customer Question:

My device is not charging. What should I do?

GROUNDED RESPONSE

According to the troubleshooting guide, check that the charging cable and power adapter are securely connected. Follow the supported troubleshooting procedure in the product documentation.

RETRIEVED SOURCE

[Relevant troubleshooting guide text]

STATUS

- ✓ Relevant context retrieved
 - ✓ Semantic search completed
 - ✓ Grounded answer generated
 - ✓ Source displayed
-

Conclusion

The RAG-Based Customer Support AI Agent provides an intelligent customer-support solution using product manuals, FAQs, troubleshooting guides and policy documents. It combines document chunking, semantic embeddings, FAISS retrieval and Gemini generation to produce grounded responses. The system also displays retrieved source context and handles out-of-domain questions without inventing unsupported information